STM32 엣지 컨트롤러 — 발표 스크립트

오프닝 — 설계 의도

```
┌──────────────────────────────────────────────────────────────┐
│                              RPi 4B                          │
│        YOLO 추론 + 스트리밍  →  CPU 50~80% 상시 점유          │
│                                                              │
│        여기에 마이크 샘플링 + 서보 PWM + I2C 센서 추가 시      │
│                              실시간성 붕괴                    │
└──────────────────────────────────────────────────────────────┘

                          ↓ 해결

┌──────────────┐                  ┌──────────────────────┐
│ RPi 4B       │ UART             │ STM32F401RE          │
│              │◄────────────────►│                      │
│ AI 추론      │                  │ HW 제어 전담         │
│ 서버 통신    │                  │ 실시간 보장          │
└──────────────┘                  └──────────────────────┘
```

([그림] 은 추후 넣을 예정입니다.)1

1. 환경 맞춤형 자율 CCTV 구동

의도

RPi에서 센서를 직접 제어하면 두 가지 문제가 있습니다.

- YOLO 추론 부하 중에 I2C 폴링까지 처리하면 응답 지연 발생
- RPi가 다운되면 센서 기반 장비 보호 로직도 같이 멈춤

STM32가 센서를 독립적으로 처리하고, 임계 조건 충족 시 RPi 명령 없이 즉각 대응합니다.

구조

```
AHT10 (온습도)        ┐
                      ├─ I2C ─▶ STM32 판단 ─▶ 히터 ON  (결로 방지)
PCF8591 (조도)        ┘                   └──────▶ IR LED ON (야간 모드)
```

[그림] 센서 → STM32 → 액추에이터 흐름도 AHT10/PCF8591 → 판단 로직 → 히터/IR LED

FreeRTOS 태스크 분리

ControlTask (10ms) ─ 모터 제어 루프

SensorTask (20ms) ─ AHT10 / PCF8591 읽기 ← 별도 태스크로 격리

I2C 응답이 늦어도 모터 제어 주기에 영향 없음

세부 동작

조건	**STM32** 즉각 동작
조도 < 임계값	IR LED 점등
습도 > 임계값	히터 ON
RPi 정상 시	`/dev/stm_env` 로 상태 읽기 가능

AHT10: 측정 주기 최소 1초 강제 + N=10 이동 평균 (너무 자주 읽으면 센서 자체 발열 → self-heating 오측정)

---

2. TDOA 기반 음원 추적

의도 **:**

- 기존 아날로그 마이크 2개로는 좌/우 이진 방향 판정만 가능했습니다.
- 카메라 FOV 밖에서 이상 음원이 발생했을 때 정확한 방향으로 PTZ를 회전시키려면 단순 이진 판정으로는 부족합니다. 도달 시간차(TDOA)를 계산해 -90°~+90° 연속 각도를 산출하고, 그 값으로 PTZ를 직접 추종하는 구조로 전환했습니다.

- TDOA를 위해 채널 간 동기화가 필수였고, ADC 순차 스캔 방식으로는 TDOA 연산 자체가 불가능해서 I2S로 전환했습니다.

[그림] ADC vs I2S 타이밍 비교 다이어그램 ← 이게 핵심, Time Skew 시각화

ADC vs I2S — 왜 바꿨나

[ADC 방식]	[I2S 방식]
L채널 샘플링	L채널 ┐
↓	├ 동일 클럭, 동시 샘플링
수십 μs 딜레이	R채널 ┘
↓	
R채널 샘플링	
→ Time Skew 발생	→ Time Skew 없음
→ 두 채널의 "시간차" 오염	→ 순수한 도달 시간차만 존재
→ TDOA 계산 자체가 불가	→ TDOA 연산 가능
→ L이냐 R이냐, 이진 판정만 가능	→ -90° ~ +90° 연속 각도 산출

[그림] TDOA 파이프라인 플로우차트 I2S → VAD → Overlap → GCC-PHAT → 각도 → 모터

I2S가 TDOA에 적합한 이유

	ADC	I2S
채널 동기화	순차 스캔 (Time Skew)	동일 클럭 동시 샘플링
버퍼 방식	CPU 직접 읽기	DMA 더블 버퍼링
해상도	12-bit	24-bit
샘플 유실	CPU 바쁘면 유실	CPU 개입 없이 연속 수신
TDOA 적합성	❌	✅

DMA 더블 버퍼링 덕분에 CPU가 YOLO 추론으로 바빠도 샘플이 끊기지 않습니다. 24-bit 해상도는 미세한 도달 시간차를 더 정밀하게 계산할 수 있게 해줍니다.

TDOA 파이프라인

I2S DMA 수신

→ DC blocking (DC_TRACK_SHIFT) + noise gate (NOISE_GATE_TH)

→ VAD (TDOA_VAD_MEANABS_TH) — 저에너지 구간 skip

→ 50% Overlap-Save — 프레임 경계 누락 방지

→ Hann window (tdoa_prepare_hann())

→ tdoa_dft_real_128() — 128pt DFT

→ tdoa_phat_corr_at_lag() — PHAT 정규화 (TDOA_PHAT_EPS)

→ tdoa_effective_lag_max() — 물리 lag 제한 (fs × d / c)

→ 3점 포물선 보간 — 서브샘플 오프셋 p

→ $\tau = (lag + p) / fs$ → $\alpha = \arcsin(\tau \times c / d)$

→ confidence hysteresis (TDOA_CONF_ON/OFF_Q8)

→ valid hold (TDOA_VALID_HOLD_MS)

→ EMA (TDOA_EMA_ALPHA_Q8) + slew (TDOA_MAX_STEP_DEG_X10)

→ motor_ctrl_track_pan_tdoa()

모터 추종 **— motor_ctrl_track_pan_tdoa()**

각도 → PWM 매핑

→ deadband (TDOA_DEADBAND_PWM) — 미세 떨림 무시

→ 가변 이득 (TDOA_GAIN_Q8) — 오차 비례 step

→ step 제한 (TDOA_STEP_MIN/MAX_PWM)

→ 가감속 제한 (TDOA_ACCEL_UP/DN_PWM)

→ 오버슈트 방지 clamp

TDOA invalid → TDOA_FALLBACK_HOLD_MS 후 detect_dir 경로로 복귀 (SRC:D)

[그림] 이진 제어 vs 연속 각도 추종 비교 Before: L/R 두 방향 After: -90° ~ +90° 연속

8단계 진화 요약

단계	버전	해결한 것
1	v1.2.6	API 인터페이스 고정
2	v1.2.7	VAD + 50% Overlap
3	v1.2.8	GCC-PHAT + 서브샘플 보간
4	v1.2.9	히스테리시스 + hold + EMA
5	v1.2.10	AUTO PAN 연속 추종 + fallback
6	v1.2.11	가변 이득 + 가속도 제한
7	v1.2.12	물리 lag 제한 + Hann 윈도우
8	v1.2.13	MANUAL 시 연산 스킵, 20ms 주기 제한

핵심 트러블슈팅 **3**개

① 프레임 경계 누락

Before: [————————][————————] 비명이 경계에 걸리면 유실

After: [————[————————]————] 50% overlap → 반드시 한 쪽에 걸림

② GCC-PHAT NaN

분모 = 0 → NaN → 방향 판정 루프 붕괴

해결: 분모에 ε (1e-10) 추가

③ 모터 진동 **(confidence** 깜빡임**)**

Before: valid → invalid → valid → invalid → PAN 진동

After: ON 임계 / OFF 임계 분리 + hold 시간 → 모터 안정

---

3. Watchdog — 시스템 자가 복구

의도

- RPi 소프트웨어 Watchdog은 커널 패닉 발생 시 OS와 함께 정지합니다. 즉, 소프트웨어 레이어에서는 복구 수단이 없습니다.
- STM32의 IWDG는 내부 LSI 클럭으로 독립 동작하기 때문에 RPi 상태에 영향을 받지 않습니다.
- 여기에 릴레이를 붙여서 RPi 주전원을 물리적으로 차단·재인가하는 Hard Reset 경로를 확보했습니다. 무인 설치 환경에서 원격 재부팅 수단 없이도 장애 복구가 가능하도록 하기 위해서입니다.

[그림] SW WDG vs HW WDG 비교 커널 패닉 시 SW는 같이 죽고, STM32 IWDG는 독립 동작

[그림] 외부 WDG 타이밍 다이어그램 PC3 신호선 + PC10 릴레이 신호선 정상 → 소실 → 10s → cut → restore

---

구조

```
                    STM32 (독립 동작)
                          │
        ┌─────────────────┼─────────────────┐
        │                 │                 │
  IWDG (내부)        RPi Heartbeat      릴레이 제어
  자기 자신 감시      PC3 감시            PC10
        │                 │                 │
  4초 무응답     →   10초 무신호      →   RPi 전원 차단
  STM32 리셋          릴레이 ON          3초 후 재인가
```

RPi OS가 완전히 죽어도 STM32 LSI 클럭은 독립적으로 동작

외부 **WDG** 흐름

```
부팅
└─ grace 60초 (RPi 부팅 대기, 오탐 방지)
   └─ PC3 첫 엣지 감지 → hb_armed = 1 → 감시 시작
      ├─ 정상: 10초 내 신호 수신 → 타이머 리셋
      └─ 이상: 10초 초과 → relay cut 3초 → 전원 재인가
                         → grace 재시작
```

실제 있었던 이슈

스택 오버플로 **(v1.2.3)**

증상: 부팅 직후 "[RPI-WDG] st" 로그 중간 절단 후 반복 재부팅

원인: defaultTask 스택 512B → vsnprintf + UART 전송 시 오버플로

→ 태스크 손상 → IWDG 갱신 중단 → 하드웨어 리셋 반복

해결: 512B → 1536B 증설

IWDG refresh 밀림 (v1.2.2)

원인: SystemMonitorTask Low 우선순위 → UART burst 시 밀림

해결: SystemMonitorTask AboveNormal 상향

UartRxTask Normal 하향

오탐 릴레이 컷 **(v1.2.4)**

원인: RPi 부팅 완료 전, 첫 heartbeat 오기 전에 timeout 판정

해결: hb_armed 플래그 도입 → 첫 엣지 감지 후에만 감시 시작

리셋 원인 불명 **(v1.2.2)**

해결: 부팅 시 RCC 플래그 출력

[RST] IWDG:1 BOR:0 PIN:0 ... → 원인 즉시 분류

---

4. 커널 드라이버 — 하드웨어 추상화

의도

- STM32-RPi 간 UART 프로토콜은 `CMD(1B) + LEN(4B big-endian) + PAYLOAD(JSON)` 바이너리 프레임입니다.
- 상위 애플리케이션 개발자가 이 프레임을 직접 조립하게 되면 하드웨어 의존성이 애플리케이션 코드에 섞이고, STM32 펌웨어 변경 시 서버 코드도 함께 수정해야 합니다.
- 커널 캐릭터 드라이버를 작성해서 UART 처리를 커널 레벨로 내리고, 상위에는 `/dev/stm_motor`, `/dev/stm_env` 같은 표준 파일 인터페이스만 노출하고, 하드웨어 변경이 애플리케이션 레이어로 전파되지 않도록 계층을 분리하였습니다.

구조

```
┌──────────────────────────────────────┐
│              서버 애플리케이션              │
│           open() / read() / ioctl()          │
└──────────────────┬───────────────────┘
                   │ 표준 파일 I/O
┌──────────────────▼───────────────────┐
│          stm_bridge.ko (커널 드라이버)        │
│                                      │
│        /dev/stm_motor  → CMD 0x04    │
│        /dev/stm_env    → CMD 0x05    │
│        /dev/stm_audio  → 마이크 이벤트       │
│        /dev/stm_ir     → IR 제어      │
└──────────────────┬───────────────────┘
                   │ UART 바이너리 프레임
┌──────────────────▼───────────────────┐
│              STM32F401RE             │
└──────────────────────────────────────┘
```

Device Tree Overlay로 UART 활성화 + serial getty 비활성화

Before / After

```
/* Before — 서버 팀이 직접 조립 */

frame[0] = 0x04;

frame[1] = 0; frame[2] = 0;

frame[3] = 0; frame[4] = len;

memcpy(frame + 5, json, len);

write(serial_fd, frame, 5 + len);


/* After — 표준 파일 I/O */

int fd = open("/dev/stm_motor", O_WRONLY);

ioctl(fd, SET_PAN_TILT, &cmd);
```

STM32 프로토콜이 바뀌어도 드라이버 내부만 수정, 서버 코드는 그대로

UART 안정화 — 핵심 수정

증상: 응답 없음 ↔ 응답 폭주 반복 / 로그: `IV/OV`만 증가 (물리 에러 아님)

수정	내용
CMD 화이트리스트	0x04 / 0x05 외 즉시 폐기
프레임 타임아웃	100ms 침묵 시 상태머신 reset
완료 프레임 링큐	슬롯 1개 → 8개
처리 경로 분리	수신(UartRxTask) / 응답(ControlTask) 분리
응답 레이트 제한	100ms 내 중복 요청 병합 → 1회 응답
큐 확장	16 → 64

마무리

```
┌──────────────────────────────────────────────┐
│                RPi가 죽어도 카메라는 살아있어야 한다   │
│                                              │
│                 환경 센서  →  자율 야간 전환 / 결로 방지 │
│                TDOA     →  사각지대 음원 추적        │
│                Watchdog  →  커널 패닉도 자가 복구      │
│                커널 드라이버  →  팀 간 완전 분리       │
└──────────────────────────────────────────────┘
```